



Emulator NES pe ESP32-S3

Proiectare cu Microprocesoare

Autor: Stolniceanu Iustin-Pavel

Grupa: 30237

FACULTATEA DE AUTOMATICA
SI CALCULATOARE

8 ianuarie 2026

Cuprins

1	Introducere	2
1.1	Prezentare Generala	2
1.2	Performanta si Compromisuri	2
2	Montaj Electric	2
2.1	Schema de Conectare	2
3	Detalii putin mai tehnice	3
3.1	Interactiunea cu Perifericele si Multithreading	3
3.1.1	Arhitectura Task-urilor	3
3.1.2	Gestionarea Afisajului	3
3.1.3	Obtinerea datelor de la controllerul de PS2	4
3.2	Sistemul de fisiere - LittleFS	4
3.3	Sincronizarea aproape perfecta	4
3.4	Simularea procesorului	5
4	Manual de Utilizare	5
4.1	Instalare si Configurare	5
4.2	Interactiunea cu Sistemul	5
5	Referinte	6

1 Introducere

1.1 Prezentare Generala

Acest proiect detaliaza implementarea unui emulator functional pentru consola Nintendo Entertainment System (NES) pe platforma embedded ESP32-S3. Proiectul reprezinta o adaptare a unui emulator scris initial pentru arhitectura x86 (dezvoltat tot de mine), optimizat acum pentru a rula pe resurse mult mai limitate (o tranzitie foarte grea de la 16 GB RAM la doar 512 KB).

Obiectivul principal este executia jocurilor originale, care vin sub forma unor fisiere ROM ce contin cod compilat pentru procesorul 6502 si date grafice, direct pe microcontroller. Sistemul ofera iesire video pe un ecran TFT si control prin intermediul unui gamepad de PlayStation 2 (PS2).

1.2 Performanta si Compromisuri

Sistemul atinge o viteza de randare de aproximativ 24 FPS in jocuri precum *Super Mario Bros.* Pentru a atinge aceasta performanta pe ESP32-S3, emularea nu este „cycle-perfect”. S-a eliminat sincronizarea cycle-perfect dintre CPU si PPU si s-a preferat un sistem in care unitatea grafica(PPU) randeaza un scanline(o linie de pixeli), iar apoi procesorul ruleaza pentru ”prinde din urma” unitatea grafica. S-au eliminat toate branch-urile din emularea procesorului, emularea procesorului fiind redusa la:

- Fetch - obtinerea unui opcode din memorie
- Decode, cu ajutorul unui lookup table
- Execute, cu ajutorul unui lookup table cu pointeri la functii

2 Montaj Electric

Hardware-ul utilizat consta intr-o placa de dezvoltare ESP32-S3, un display TFT bazat pe driverul ST7789 si un controller PS2 conectat prin interfata SPI software. Hardware-ul utilizat:

1. ESP32-S3
2. Controller PS2 original
3. Display TFT cu driverul ST7789, de rezolutie 256x240

2.1 Schema de Conectare

Deoarece biblioteca standard SPI a intampinat limitari in aceasta configuratie, comunicarea cu controllerul PS2 se realizeaza prin „bit-banging” pe pini GPIO specifici. Conexiunile sunt definite astfel:

Semnal PS2	Pin ESP32	Descriere
DAT (Data)	GPIO 40	Linie de date (MISO)
CMD (Command)	GPIO 39	Linie de comanda (MOSI)
ATT (Attention)	GPIO 38	Chip Select (CS)
CLK (Clock)	GPIO 37	Clock
VCC	3.3V	Alimentare
GND	GND	GND

Tabela 1: Maparea pinilor pentru Controllerul PS2

Mai jos este prezentata schema electrica realizata in Fritzing, care ilustreaza interconectarea modulelor.

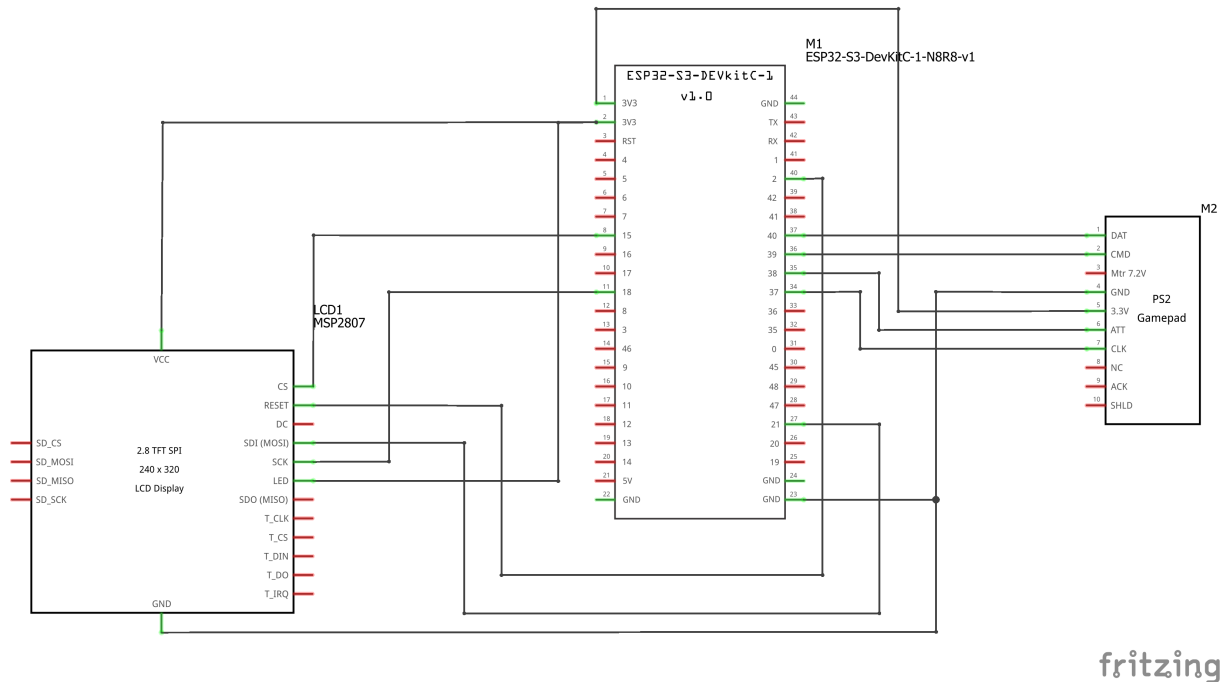


Figura 1: Schema electrica a sistemului (ESP32, TFT, PS2)

3 Detalii putin mai tehnice

3.1 Interactiunea cu Perifericele si Multithreading

Microcontrollerul ESP32-S3 dispune de doua nuclee (Core 0 si Core 1), caracteristica exploatata pentru a paraleliza sarcinile critice folosind FreeRTOS.

3.1.1 Arhitectura Task-urilor

Aplicatia este divizata in trei fire de executie principale:

- **Core0Loop (Nucleul 0):** Ruleaza logica intensiva de emulare (CPU si PPU). Este izolat pe acest nucleu pentru a evita intreruperile cauzate de alte procese.
- **ps2Task (Nucleul 1):** Un task dedicat cu prioritate ridicata care interogheaza starea controllerului PS2 la fiecare 10ms.
- **Main Loop (Nucleul 1):** Gestioneaza logica meniului, incarcarea jocurilor si afisarea efectiva a imaginii pe ecran (prin biblioteca TFT_eSPI).

3.1.2 Gestionarea Afisajului

Display-ul este initializat folosind biblioteca TFT_eSPI . Rezolutia interna a NES este de 256x240 pixeli. Pixelii sunt stocati intr-un **frameBuffer** (Sub formatul RGB565 - 5 biti pentru Rosu, 6 pentru verde, 5 pentru albastru) si trimisi catre ecran doar atunci cand s-a generat un frame complet.

3.1.3 Obținerea datelor de la controllerul de PS2

Cand ma refer la "PS2", ma refer la Playstation2, nu portul pentru tastaturi de pe calculatoarele antice, care aveau performante similare cu cele ale acestui ESP32-S3 :)). Controller-ul de PS2 comunica prin SPI, deci e foarte trivial sa obtinem informatii de la el.

```
void PS2::readController()
{
    digitalWrite(PS2_ATT, LOW); //PS2-ATT is just Chip-select!

    shiftInOut(0x01);
    shiftInOut(0x42);
    shiftInOut(0x00);
    buttonsLow = shiftInOut(0x00);
    buttonsHigh = shiftInOut(0x00);

    digitalWrite(PS2_ATT, HIGH);
}
```

3.2 Sistemul de fisiere - LittleFS

Fisierele ROM cu jocuri sunt incarcate pe ESP32-S3 cu ajutorul sistemului de fisiere LittleFs. Aceasta librerie si acest sistem de fisiere ofera access la apeluri de sistem foarte similare cu cele din Linux, pentru read si write binar din fisier.

3.3 Sincronizarea aproape perfecta

Sincronizarea perfecta la nivel de ciclu este imposibila pe un hardware care ruleaza mai incet decat un 486 :))) . Solutia adoptata este randarea per scanline(scanline vine de la CRT-urile vechi, se refera doar la o linie de pixeli cu inaltimea 1). PPU randeaza o linie completa, care stim ca dureaza exact 113.66666 cicluri de procesor, iar CPU executa numarul echivalent de instructiuni pentru a „prinde din urma” timpul scurs. Se foloseste un mecanism de *Leap Cycle* pentru a corecta diferentele de timing, deoarece raportul dintre frecventele PPU si CPU nu este un numar intreg perfect.

```
inline void bus_clock_t()
{
    // PPU randeaza o linie de scanare (341 cicluri PPU)
    ppu_render_scanline();

    // CPU executa instructiuni pentru a compensa timpul (~113 cicluri)
    // Se adauga un ciclu suplimentar (leap_cycle) periodic
    for (int i = 0; i < 113 + leap_cycle; i++)
    {
        cpu_clock();
        leap_cycle != leap_cycle; // Toggle pentru corectie
    }
}
```

3.4 Simularea procesorului

Este impropriu spus sa spun ca asta e un emulator. Nefiind 1-1 cu hardware-ul original, e mai mult un simulator. Ce am dezvoltat eu pe x86 poate fi numit emulator, intradevar, dar asta e mai degraba un simulator. Mai jos prezint functia care ruleaza procesorul pentru exact un ciclu de ceas:

```
void IRAM_ATTR cpu_clock()
{
    if (remaining_cycles)
    {
        remaining_cycles--;
    }
    else
    {
        if (!pending_nmi)
        {
            opcode = read_pc();
            remaining_cycles += OPCODE_duration[opcode];
            opcode_table[opcode](); //this runs the corresponding instruction
        }
        else
        {
            trigger_nmi();
            remaining_cycles += 7;
            return;
        }
    }
    return;
}
```

6502 este un procesor multi-cycle, asa ca, pentru a "taia colturi", am ales sa simulez instructiunea, apoi sa astept pentru atatea cicluri de ceas cat este necesar. Asta simplifica foarte mult logica si face posibila obtinerea unui frame-rate decent pe un hardware atat de limitat.

4 Manual de Utilizare

4.1 Instalare si Configurare

1. Se cloneaza repository-ul meu. Trebuie sa aveti PlatformIO instalat.
2. Se copiaza fisierele .nes (ROM-uri) in directorul data al proiectului.
3. Se incarca sistemul de fisiere pe placa ESP32: `pio run --target uploadfs`.
4. Se compileaza si se incarca codul sursa pe EPS32-S3.

4.2 Interactiunea cu Sistemul

La pornire, utilizatorul este intampinat de un meniu grafic simplificat, care listeaza jocurile disponibile pe memoria flash.



Figura 2: Interfata meniului principal pentru selectia jocurilor

Comenzi in Meniu:

- **D-PAD:** Navigare sus/jos in lista de jocuri.
- **X (Cross):** Selecteaza si lanseaza jocul evidentiat.

Comenzi in Joc:

- **D-PAD / Butoane:** Functionalitate standard NES.
- **L2:** Revenire la meniul principal (Resetare).
- **Cerc (O):** Activare mod Debug/Profilare (afiseaza FPS si cicluri in consola seriala).

5 Referinte

1. <https://www.nesdev.org/> - THE NES BIBLE
2. <https://llx.com/Neil/a2/opcodes.html> - Instructiuni 6502
3. <https://www.nesdev.org/obelisk-6502-guide/> - 6502 Guide
4. <http://www.billporter.info/2010/06/05/playstation-2-controller-arduino-library-v1-0/> - biblioteca pe care am facut reverse engineering pentru a afla cum se interactioneaza cu controllerul de PS2 + explicatie pini controller PS2
5. <https://github.com/stolniceanuiustin/NESEmulatorCPP> - emulatorul pentru x86
6. <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/performance/speed.html> - pentru optimizarea codului